

PHP fortsättning

Förutsättningar för att följa guiden

Formulär

Filhantering (mode)

Öppna och läsa fil (fopen, fread)

Skriva till fil (fwrite)

Öppna och läsa fil (file, file_get_contents)

Radbrytning (nl2br)

Öppna och skriva till fil (file_put_contents)

Infoga fil (include, require)

Sessioner

Cookies

nästa steg: MySQL och databaser ->

Förutsättningar för att följa guiden

För att du ska kunna följa denna guide förutsätter vi att du har **grundkunskaper** i PHP motsvarande guiden [PHP-grunder »](#)

OBS! Förutsättningar för att följa guiden: alla exempel som visas i denna guide använder PHP installerat lokalt och Apache webserver bearbetar koden innan den visas i webbläsaren. Den lokala rotmappen är **C:\www** om du installerat Apache och PHP enligt guiden ["installera PHP" »](#) och adressen till din lokala webserver är då **http://localhost** eller **http://127.0.0.1**

Formulär

PHP kan användas för att bearbeta innehållet som anges i formulär och skicka eller hämta information (här visas inte hur du skapar de olika formulärkontrollerna vilket du kan läsa mer om i guiden ["Formulär" »](#)).

För att få formuläret att fungera behöver du två filer; formuläret skapas i dokumentet **order.htm** och innehållet bearbetas och visas i dokumentet **process.php**

1. Vårt exempel är en minishop där det bara finns tre artiklar i tre färger och där kundens namn anges med för- och efternamn. Som action för formuläret anges att innehållet i fälten ska skickas till filen "process.php" för att bearbetas:

```
<form action="http://localhost/process.php" method="post">
```

De **namn** du använder på dina formulärkontroller kommer senare att användas som **nycklar** i filen "process.php" och det är därför viktigt att du använder samma versaler/gemenor i namnen på i båda dokumenten.

HTML-koden för att skapa formuläret som du kan spara med namnet **order.htm** (i din lokala rotmapp C:\www):

```

<html>
<head><title>Order</title></head>
<body>
<h2>Beställningsformulär</h2>

<form action="http://localhost/process.php" method="post">
Artikel: <select name="artikel">
<option selected>Välj artikel här:</option>
<option>Byxa</option>
<option>Tröja</option>
<option>Skor</option>
</select>
Antal: <input name="antal" type="text" size="5">
Färg: <select name="color">
<option selected>Välj färg här</option>
<option>Röd</option>
<option>Grön</option>
<option>Blå</option>
</select> <br>
<h4>Kunduppgifter </h4>
Förnamn: <input name="fnamn" type="text">
Efternamn: <input name="enamn" type="text"> <br>
<input name="skicka" type="submit" value="Skicka beställning">
</form>

</body>
</html>

```

HTML-dokumentet **order.htm** som det kan se ut när det öppnats i webbläsaren:



(Formuläret fungerar inte förrän du skapat PHP-dokumentet i nästa steg)

2. Filen **process.php** som anropas av formuläret består av **vektorer** (arrays) där värden från formuläret anges som värden i vektorn (läs mer om arrays i avsnittet "[Vektorer](#)" »). Varje vektor som ska visas placeras i en variabel. Vektorn innehåller en nyckel som har samma namn som namnet på formulärkontrollerna:

```
$artikel = $_POST["artikel"];
```

PHP-koden skriver ut innehållet som angivits i formulärfälten som variabler tillsammans med text och radbrytningar:

```

<html>
<body>

```

```
<?php

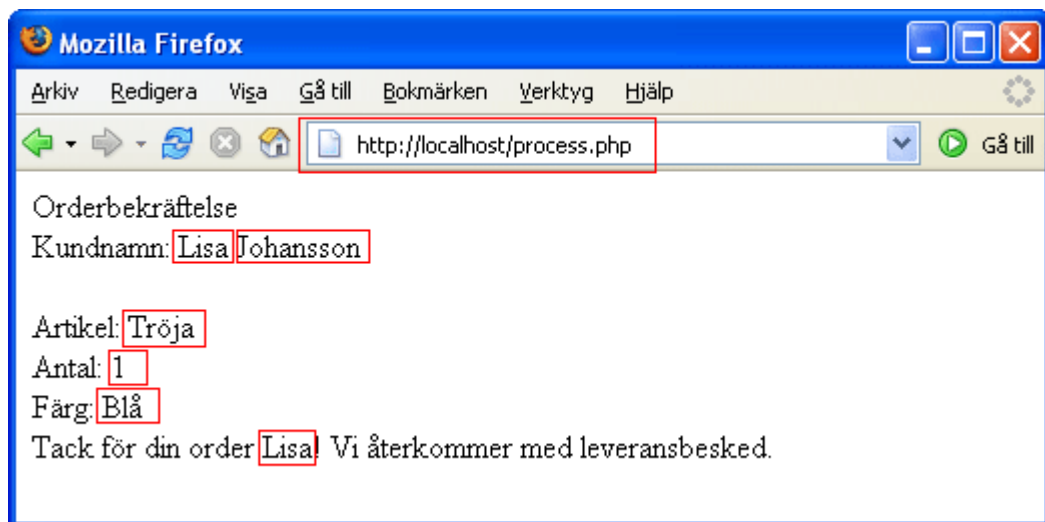
$artikel = $_POST["artikel"];
$antal = $_POST["antal"];
$color = $_POST["color"];
$fnamn = $_POST["fnamn"];
$enamn = $_POST["enamn"];

echo "Orderbekräftelse <br />";
echo "Kundnamn: " . $fnamn . " " . $enamn . "<br />";
echo "Du har beställt följande:" . "<br />";
echo "Artikel: " . $artikel . "<br />" . "Antal: " . $antal . "<br />" . "Färg: " . $color .
"<br />";
echo "Tack för din order " . $fnamn . " ! Vi återkommer med leveransbesked.";

?>

</body>
</html>
```

När PHP-dokumentet "process.php" sparats och formuläret i "order.htm" skickats bearbetas och visas resultatet av beställningen:



[Klicka här »](#) för att prova exemplet ovan. Prova att fylla i uppgifterna och skicka iväg beställningen!

Filhantering

Först några ord om att du bör vara **försiktig** när du arbetar med filhantering i PHP. Du kan skapa oreda i filer och din dator om du redigerar eller raderar fel filer. Även om du har koll på dina filnamn och sökvägar så bör du ändå vara medveten om att det finns en viss risk. Tanken med de här avsnitten om filhantering är att du ska få en bra grund och undvika de misstagen när du jobbar med filhantering i PHP så tveka alltså inte att lära dig mer om filhantering!

När du öppnar en fil kan du bestämma om den ska vara **läsbar**, **skrivbar**, eller både läs- och skrivbar. Dessutom väljer du om innehållet i den öppnade filen ska finnas kvar eller skrivas över och var filpekaren ska placeras. **Filpekaren** är för PHP en position i filen men kan jämföras med textmarkören när du skriver text, där markören är placerad skrivs ju även den nya texten.

Funktionerna **fopen**, **fread**, **fwrite** och **fclose** är de som huvudsakligen används för att öppna, läsa, skriva och stänga filer. Parameter för läget (**mode**) anger du när filen öppnas och tabellen nedan visar vilka mode du kan ange:

Mode	Beskrivning
r	Öppnar för läsning , med start i början av filen. (r = read)
r+	Öppnar för läsning och skrivning , med start i början av filen.
w	Öppnar för skrivning . Om filen inte finns skapas den först. Om filen finns skrivs allt innehåll över med start i början av filen (tidigare innehåll raderas). (w = write)
w+	Öppnar för läsning och skrivning . Om filen inte finns skapas den först. Om filen finns skrivs allt innehåll över med start i början av filen (tidigare innehåll raderas).
a	Öppnar för skrivning . Om filen inte finns skapas den först. Om filen finns skrivs inte innehållet över. Nytt innehåll skrivs i slutet av filen. (append = lägga till)
a+	Öppnar för skrivning och läsning . Om filen inte finns skapas den först. Om filen finns skrivs inte innehåll över. Nytt innehåll skrivs i slutet av filen.

Öppna och läsa fil (**fopen**, **fread**)

Att öppna en fil förutsätter att filen finns och i PHP används samma kommando för att **skapa** en fil som att **öppna** en fil. Funktionen **fopen** öppnar en fil men om den inte existerar så skapas en fil med det angivna namnet (om du angivit att den ska vara skrivbar med mode **w**). Om du angivit mode **r** så öppnas filen för läsning (om filen redan existerar).

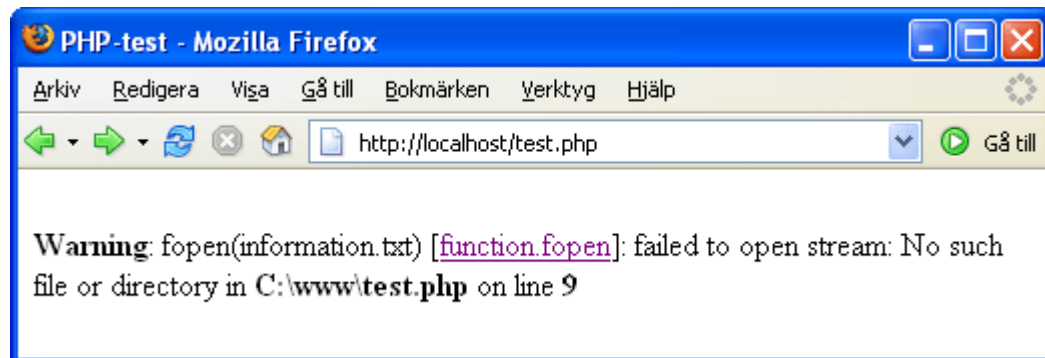
1. Exemplet nedan **öppnar en fil för läsning** (mode **r**). Filen med namnet "information.txt" förutsätts ligga i samma mapp som din PHP-fil (den lokala rotmappen C:\www i vårt exempel).

```
<?php  
  
$file = fopen("information.txt", "r");  
  
?>
```

Variabeln **\$file** skapas för att innehålla argumentet **fopen** och vilket läge (mode) filen ska öppnas i. Variabelnamnet bestämmer du själv och det kunde lika gärna vara **\$fil**, **\$dokument**, **\$filnamn** eller annat namn du tycker är passande.

Variabeln **\$file** kan även kallas för vårt "filhandtag" (filehandle) som använd för att komma åt filen.

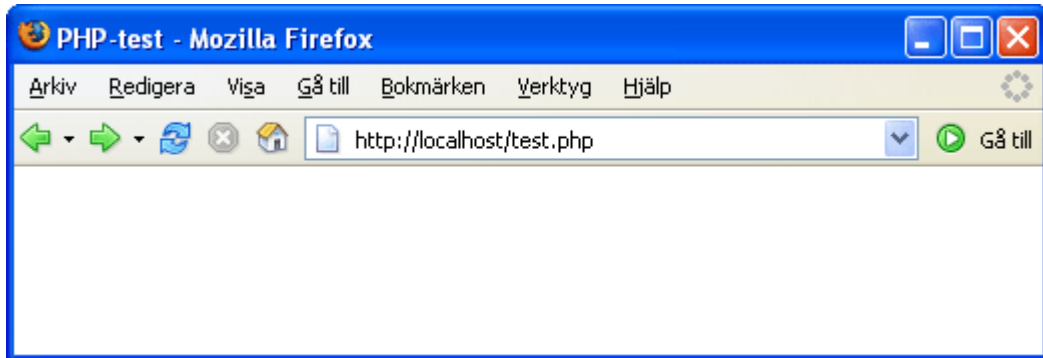
Om filen **inte finns** så kan den inte öppnas för läsning och ett felmeddelande visas av PHP:



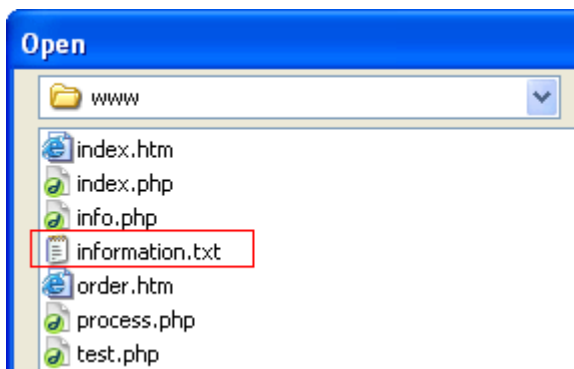
2. Om du istället öppnar samma filnamn för **skrivning** (mode **w**) så **skapas** en fil med det angivna namnet om den inte redan existerar:

```
<?php  
  
$file = fopen("information.txt", "w");  
  
?>
```

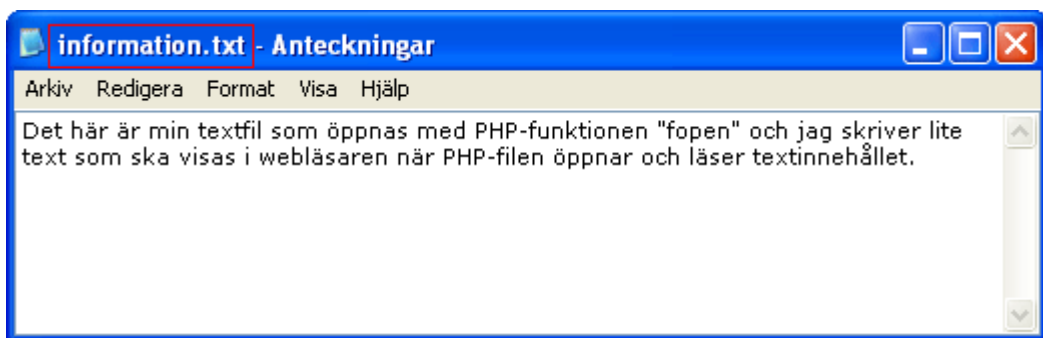
Filen "information.txt" **skapas** och **öppnas** men inget visas i webbläsaren:



3. **Öppna** filen "information.txt" som skapades i samma mapp som din PHP-fil:



Filen innehåller ingen information men du kan nu **skriva in egen text** som kan användas i kommande exempel. Här används Notepad/Anteckningar för att skriva texten:



4. För att **öppna** använder du funktionen **fopen** igen men nu med mode **r** för att inte skriva över det innehåll som redan finns i textfilen. OBS! Om du använder mode **w** kommer all din text att skrivas över och försvinna!

För att **läsa** filen används funktionen **fread** som läser innehållet och definieras i en egen variabel (här med namnet `$innehall`). Dessutom anges **antal byte** (tecken) som ska läsas in. För utskrift av innehållet används som i tidigare exempel **echo** tillsammans med variabeln `$innehall` (som nu innehåller den inlästa texten):

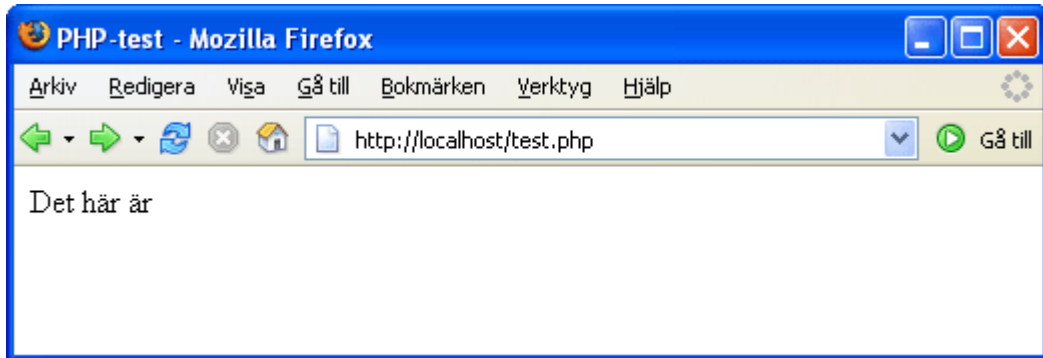
```
<?php

$file = fopen("information.txt", "r");
$innehall = fread($file, 10);

echo $innehall;

?>
```

Här visas bara de **10 första tecknen** då antalet byte angivits till 10:



5. Om du vill skriva ut **hela innehållet** i en fil måste PHP veta hur många byte filen innehåller och du använder funktionen **fread** för att returnera filstorleken i angiven fil.

Här lägger vi dessutom till funktionen **fclose** för att **stänga** filen när körningen av koden är utförd. Notera att webservern stänger filen automatiskt när körning av koden utförts men det anses ändå vara "standard" att som programmerare stänga filer när de inte ska används längre.

```
<?php

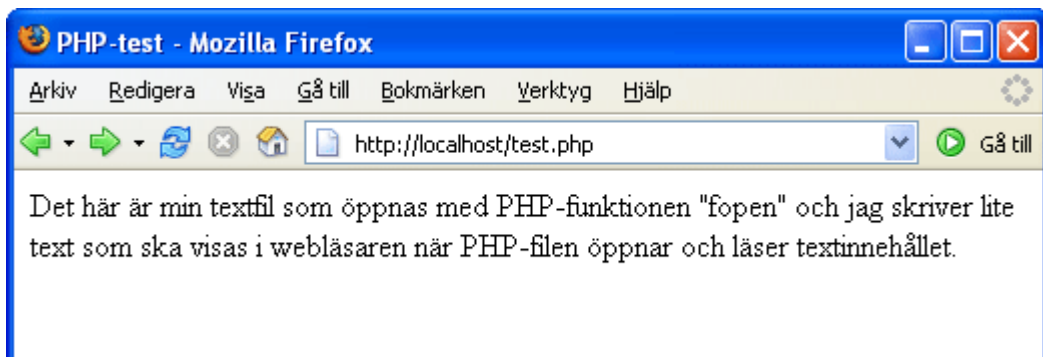
$file = fopen("information.txt", "r");
$innehall = fread($file, filesize("information.txt"));

echo $innehall;

fclose($file);

?>
```

Hela filinnehållet hämtas nu ur filen "information.txt" och skrivs ut:



Använd **fler variabler** för att göra din kod mer flexibel! I guiden [PHP-grunder](#) » angav vi att "en variabel är en behållare för information" och genom att ange filnamnet som en egen variabel kan du enkelt byta ut filens namn utan att behöva ändra på flera ställen i din kod. En alternativ lösning till kodexemplet ovan är att ange filen "information.txt" i variabeln **\$filnamn** och du anger då bara filens namn en enda gång i koden vilket är en fördel när PHP-koden senare blir mer omfattande.

Här visas två kodavsnitt som utför samma sak men där Alternativ 1 är mer flexibel:

```
<?php

//Alternativ 1: variabel med filnamn som kan bytas ut och där variabeln används i funktionerna

$filnamn = "information.txt";
$file = fopen($filnamn, "r");
$innehall = fread($file, filesize ($filnamn));
echo $innehall;
fclose($file);

//Alternativ 2: här måste filnamnet anges i varje ny funktion

$file = fopen("information.txt", "r");
$innehall = fread($file, filesize ("information.txt"));
echo $innehall;
fclose($file);

?>
```

Skriva till fil (**fwrite**)

Funktionen **fwrite** kan användas för att skriva till en fil (som redan öppnats med **fopen** enligt föregående avsnitt).

1. Vi använder samma grundkod som i Alternativ 1 ovan (filnamn som egen variabel) men tar bort avsnittet där innehållet läses in och visas på skärmen (från variabeln \$innehall). Koden nedan öppnar filen men läser den inte:

```
<?php

$filnamn = "information.txt";
$file = fopen($filnamn, "r");

fclose($file);

?>
```

2. För att **skriva** till den angivna filen använder du **fwrite** tillsammans med filhandtaget och den nya texten. OBS! För att kunna skriva till filen måste du dessutom byta **läge** till mode **w**:

```
<?php

$filnamn = "information.txt";
$file = fopen($filnamn, "w");

fwrite($file, " Jag gillar PHP!");

fclose($file);

?>
```

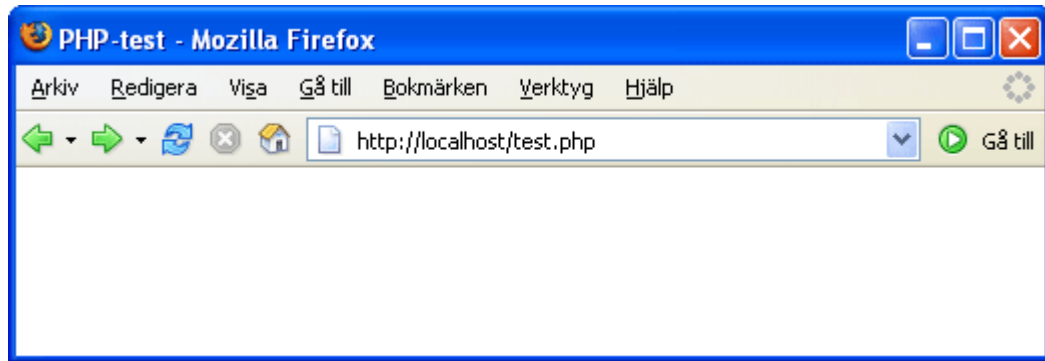
3. ...och ännu bättre blir det om du istället anger den nya texten som en egen **variabel**. Här placeras texten i variabeln **\$nytext**:

```
<?php

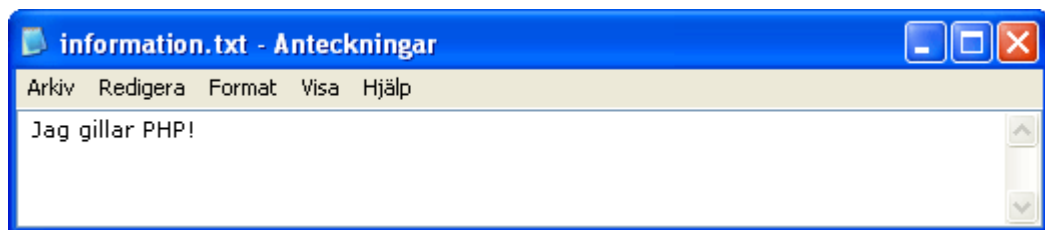
$filnamn = "information.txt";
```

```
$file = fopen($filnamn, "w");  
  
$nytext = " Jag gillar PHP!";  
fwrite($file, $nytext);  
  
fclose($file);  
  
?>
```

Texten skrivs till filen "information.txt" som öppnas (och skapas om den inte fanns förut) men inget visas i webbläsaren:



Öppna textfilen "information.txt" och kontrollera att den nya texten skrivits in:



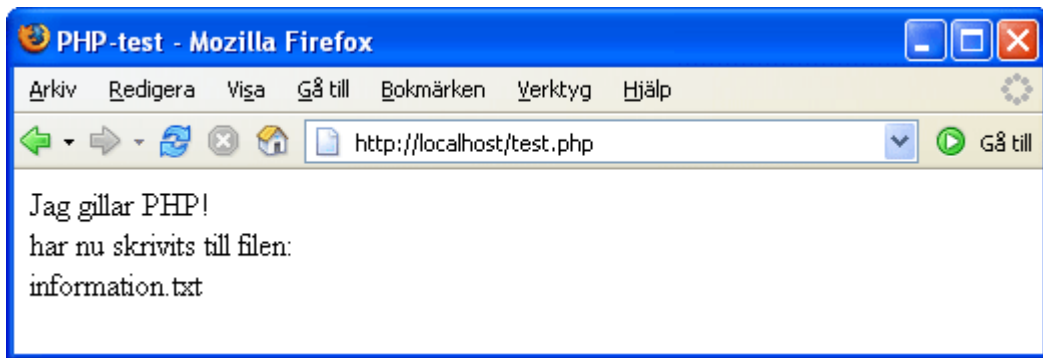
Om du följt det tidigare avsnittet så hade du redan ett textinnehåll i filen men det har nu skrivits över eftersom du angavit mode **w**.

4. För att kontrollera om texten verkligen skrivs till filen lägger vi till en **villkorssats** som skriver ut ett meddelande i webbläsaren:

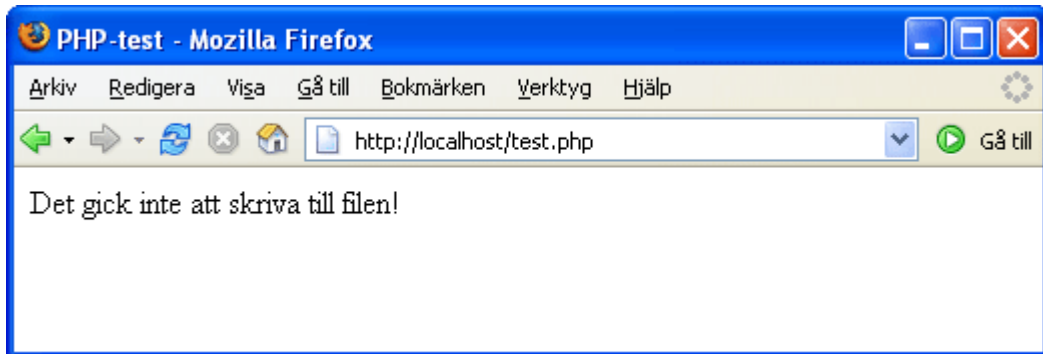
```
<?php  
  
$filnamn = "information.txt";  
$file = fopen($filnamn, "w");  
$nytext = " Jag gillar PHP!";  
fwrite($file, $nytext);  
  
if (fwrite ($file, $nytext)) {  
    echo "$nytext <br /> har nu skrivits till filen: <br /> $filnamn";  
}  
else {  
    echo "Det gick inte att skriva till filen!";  
}  
  
fclose($file);  
  
?>
```

Läs mer om **villkorssatser** i guiden ["PHP-grunder" >](#)

Om texten skrivits till filen uppfylls villkoret **if**:



Om texten inte skrivits till filen uppfylls villkoret **else**:



Prova att köra PHP-koden med mode **r** istället för mode **w** för att se felmeddelandet ovan:

```
$file = fopen($filnamn, "r");
```

Öppna och läsa fil (`file`, `file_get_contents`)

Funktionen **file** är ett bra alternativ istället för att använda **fopen**, **fread** och **fclose** för att öppna och läsa innehållet i en fil. Filen öppnas och läses in i en vektor - **array** (läs mer om arrays i avsnittet "[Vektorer](#)" ») och varje **element** i vektorn innehåller sedan **en rad** från filen. Hela filen läses in i vektorn och du behöver alltså inte ange filens storlek i byte och du behöver inte heller stänga filen, allt sker i ett och samma moment!

För att **visa** innehållet i vektorn använder du slingan **foreach** för att iterera över alla element i vektorn.

1. Vi använder **file** för att läsa in ett recept från en textfil där varje rad läses in som ett element i en array. Slingan **foreach** itererar över alla element (rader) och skapar variabeln **\$rad** som innehåller textraderna från receptet. Textfilen som används i exemplet kan du hämta här: [thai-mat.txt](#) »

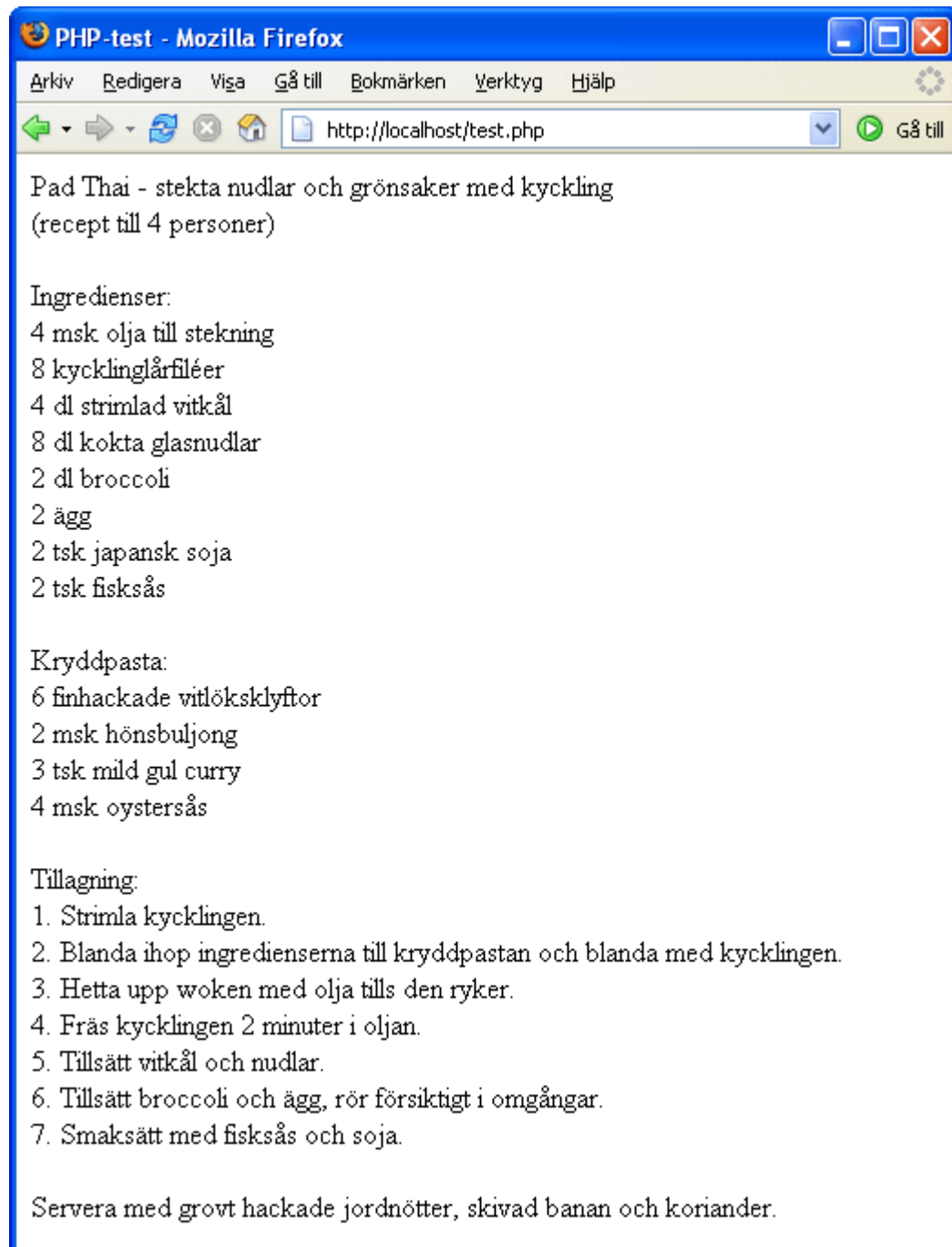
```
<?php

$filnamn = "thai-mat.txt";
$innehall = file($filnamn);

foreach ($innehall as $rad) {
    echo "$rad <br />";
}

?>
```

Receptet på den smaskiga thai-rätten läses in och visas i webbläsaren:



Funktionen **file_get_contents** är ny i PHP 4.3.0 och PHP 5.0 och är den metod som rekommenderas för att läsa in hela filen i en och samma **sträng**. Här hämtas hela filens innehåll i sin helhet till skillnad från **file** som läser i rad för rad till en array.

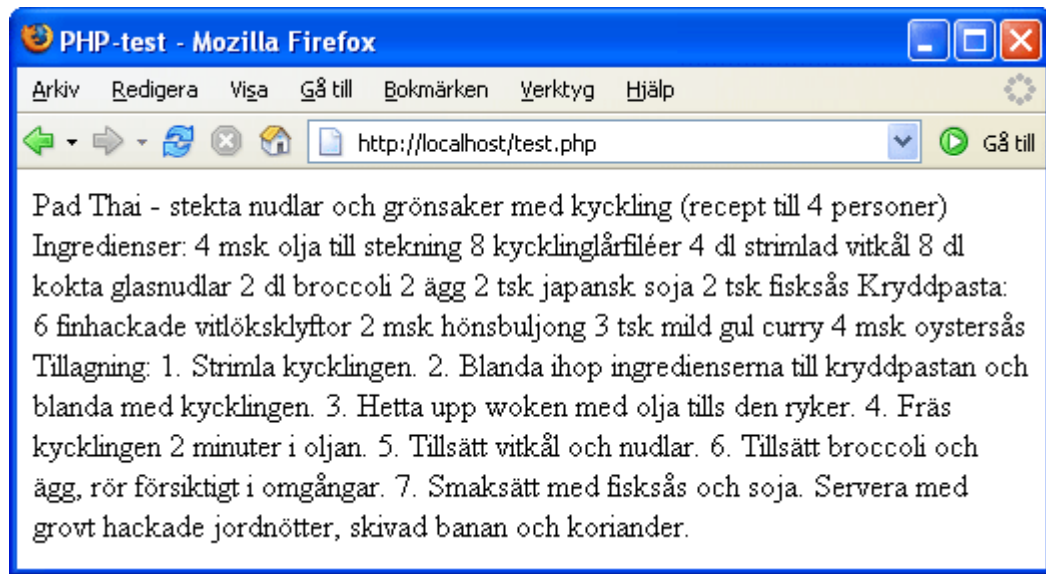
1. Textfilen med receptet hämtas in med **file_get_contents** och placeras i variabeln **\$innehall** som sedan skrivs ut i sin helhet:

```
<?php
$filnamn = "thai-mat.txt";
$innehall = file_get_contents($filnamn);

echo $innehall;
```

?>

Hela textfilen visas utan radbrytningar i webbläsaren:



Radbrytning (`<br />`)

En radbrytning i HTML skrivs som `<br>` eller `<br />` om du vill vara XML-validerad. Textfiler och även formulärkontroller i HTML som texttrutor (textarea) har radbrytningar kodade som `\n` och det gäller även för Linux/Unix-filer. Textfiler i Windows (och DOS) använder istället `\r\n` som radkod men det rekommenderas ändå att du använder `\n` för alla radbrytningar eftersom PHP ofta körs på Linux/Unix-servrar.

1. Textfilen i vårt exempel innehåller ju radbrytningar gjorda i textfilen men de visas inte i webbläsaren som inte förstår vad `\n` eller `\r\n` betyder. Om du "visar källan" till PHP-filen i webbläsaren så visar **källkoden** att radbrytningarna ändå finns när de hämtats från **textfilen**:



2. För att göra om texfilens radbrytningar till HTML-radbrytningar så kan du använda funktionen **nl2br** (new line to break) som omvandlar `\n` till `<br />` och lägger till en radbrytning framför alla rader:

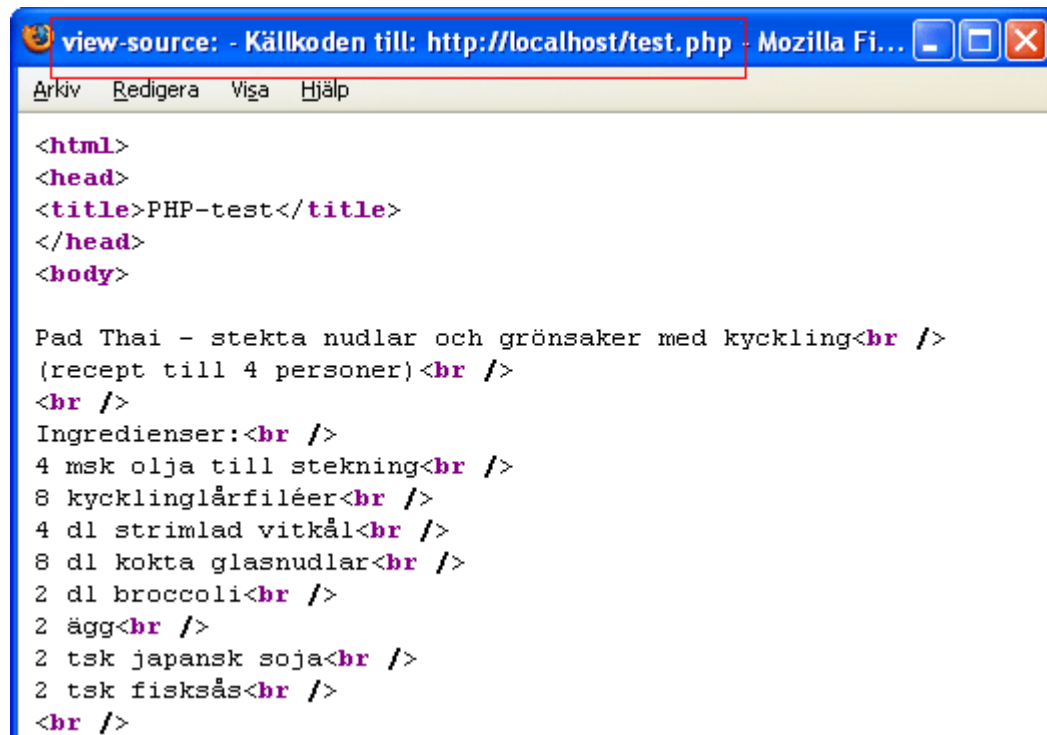
```
<?php

$filnamn = "thai-mat.txt";
$innehall = file_get_contents($filnamn);

echo nl2br($innehall);

?>
```

Resultatet är att varje rad som lästs in från textfilen får en radbrytningskod som syns i **källkoden**:



...och resultatet i webbläsaren är nu detsamma som i textfilen som hämtat med `file_get_contents`

Öppna och skriva till fil (`file_put_contents`)

En ny funktion är `file_put_contents` som ger dig möjlighet att direkt skriva till en fil utan att öppna eller stänga filen. Funktionen är ny och finns med i PHP 5.0 och framåt. OBS! Jämför gärna exemplet nedan med det tidigare avsnittet om att skriva till fil med funktionen `fwrite`.

1. `file_put_contents` skapar filen om den inte finns sedan tidigare, om filen redan finns skrivs allt innehåll över. Resultatet är detsamma som i tidigare exempel med `fwrite` (men här används bara en enda funktion):

```
<?php

$filnamn = "information.txt";
$nytext = " Jag gillar verkligen PHP!";

file_put_contents($filnamn, $nytext);

?>
```

För att kontrollera om texten verkligen skrivs till filen lägger vi till en **villkorssats** som skriver ut ett meddelande i webbläsaren:

```
<?php

$filnamn = "information.txt";
$nytext = " Jag gillar verkligen PHP!";
file_put_contents($filnamn, $nytext);

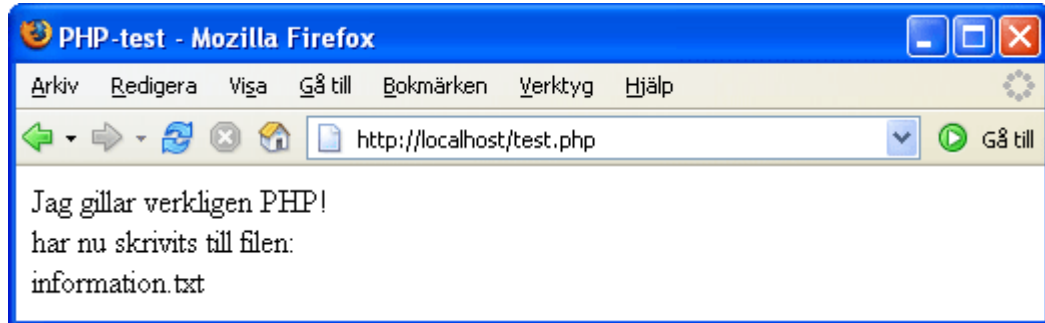
if (file_put_contents ($filnamn, $nytext)) {
```

```

    echo "$nytext <br /> har nu skrivits till filen: <br /> $filnamn";
}
else {
    echo "Det gick inte att skriva till filen!";
}

?>

```



Sammanfattning av formulär, skriva till fil och villkorsatser

Formulärfält fyll i och skickas, filer öppnas, skrivs och stängs och information skrivs även till skärmen. Du har nu lärt dig de grundläggande funktionerna för filhantering och formulär och nu kan du använda dina kunskaper i ett sammanhängande exempel.

Medlemsregistret som du använde för att läsa från en vektor (array) i slutet av guiden "PHP grunder" kan vi använda igen men nu för att skriva in medlemmarna i ett formulär som sparar medlemsnamn i en textfil. När du vill se vilka medlemmar som lagts till visar du innehållet genom att öppna och läsa textfilen och allt sker i webbläsaren med hjälp av PHP! Filerna som används är "medlemslista.txt" (som skapas automatiskt), "medlemmar.php" och "medlemslista.php" som du skapar själv.

1. Här är koden till **medlemmar.php** som innehåller både formuläret och PHP-koden för att skriva innehållet till textfilen "medlemslista.txt". (Om du är osäker på funktionerna som används kan du gå tillbaka och läsa tidigare avsnitt i PHP-guiderna).

Villkorssatsen **if** använder funktionen **empty** (tom) tillsammans med växeln **!** och **!empty** betyder då "inte tom". Filnamnet "medlemslista.txt" och funktionen **\$_POST** är placerade i egna variabler och öppnas, skrivs och stängs. Villkorssatserna **if** och **elseif** kontrollerar att filstorleken, **fsize** är större än 0 byte och skriver då meddelande till skärm. Här är hela koden till formuläret:

```

<html>
<head>
<title>Medlemmar</title>
</head>
<body>
<h2>Medlemmar</h2>

<form method="post" action="medlemmar.php">
Ny medlem: <br>
<input type="text" name="namn">
<input type="submit" value="Lägg till">
</form>

<?php

//Om formulärfältet INTE är tomt öppnas, skrivs och stängs textfilen

if(!empty($_POST["namn"])) {
    $filnamn = "medlemslista.txt";

```

```
$medlem = $_POST["namn"] . "\r\n";
$file = fopen($filnamn, "a");
fwrite($file, $medlem);
fclose($file);
}

//Om filstorleken är större än 0 byte visas strängen med bekräftelsemeddelandet
if(filesize($filnamn) > 0) {
    echo "Ny medlem är registrerad i <strong>" . $filnamn . "</strong>! <br />";
    echo "Storlek på medlemsregistret är nu " . filesize($filnamn) . " byte totalt";
}

//Endast om något skrivits i formulärfältet visas felmeddelandet
else if(!empty($_POST["namn"])) {
    echo "Det gick inte att skriva till filen!";
}

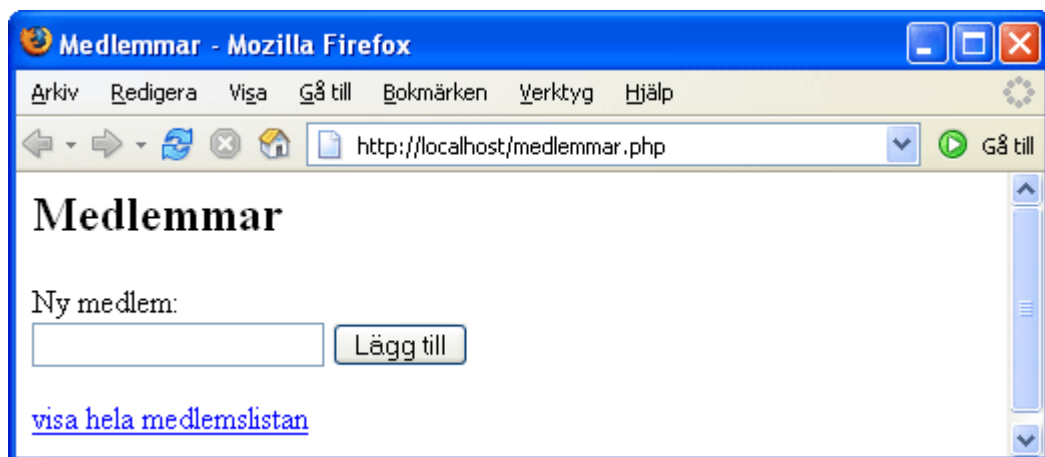
?>

<!-- länk till sidan som visar medlemmarna i textfilen -->

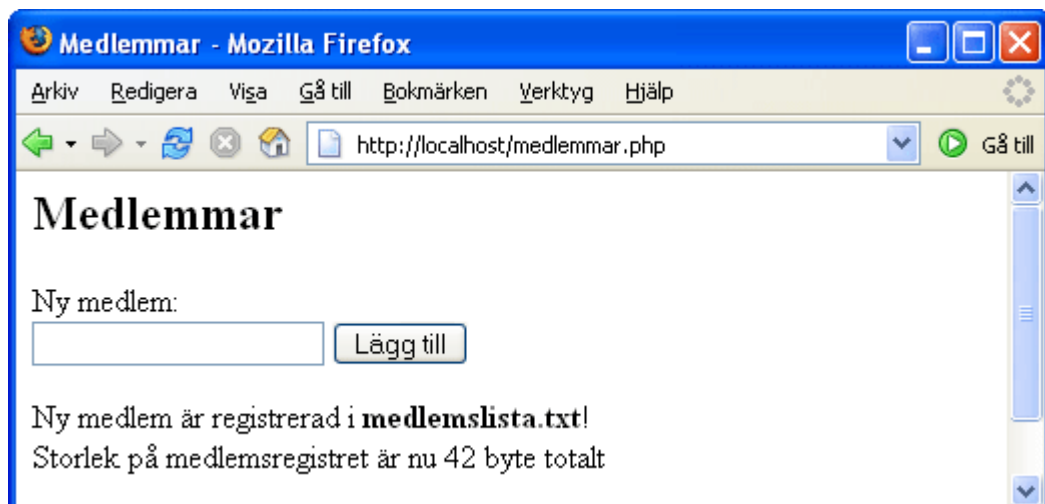
<p><a href="medlemslista.php">visa hela medlemslistan </a></p>

</body>
</html>
```

Formuläret med textfält där du kan ange de medlemmar som ska vara med:



När du registrerat en ny medlem visas bekräftelsemeddelandet med storlek på textfilen:



Registrera några fler medlemmar!

2. Länken i filen "medlemmar.php" ovan går till **medlemslista.php** som använder de båda funktionerna **file** och **file_get_contents** för att visa skillnaden mellan att läsa in filen radvis i en **array** och att läsa in hela filens innehåll som en **sträng**:

```
<html>
<head>
<title>Medlemslista</title>
</head>
<body>
<h2>Medlemslista</h2>

<p>Här läses filen in radvis i en vektor (array):</p>

<?php

//Filens innehåll läses in radvis i en array

$filnamn = "medlemslista.txt";
$innehall = file($filnamn);
foreach ($innehall as $namn) {
    echo "$namn <br />";
}

?>

<p>Samma resultat men här läses hela filen in som en enda sträng:</p>

<?php

//Filens hela innehåll läses in i en sträng

$filnamn = "medlemslista.txt";
$innehall = file_get_contents($filnamn);
echo nl2br($innehall);

?>

<!-- länk tillbaka till sidan med formuläret -->

<p><a href="medlemmar.php">tillbaka till formulärsidan</a></p>

</body>
</html>
```

När du lagt till dina medlemmar och klickar på länken visas innehållet i textfilen "medlemslista.txt" två gånger men med olika metoder:



Prova även att gå tillbaka till det länkade formuläret och lägga till fler medlemmar!

Infoga fil (**include**, **require**)

Med funktionen **include** kan du infoga en fil i koden som då även kan vara HTML-kod (om sidan sparas med filtillägget .php). Det här är användbart för menyer, sidhuvud/sidfot annan information som ska återkomma på många sidor i din webbplats. När du behöver ändra informationen gör du det bara i den fil du infogat men ser ändå resultatet på alla de sidor som visar den inkluderade filen.

Exemplet visar hur du kan infoga en **meny** som används på varje sida i din webbplats. OBS! Sidan med HTML-kod måste sparas med filtillägget **.php** för att vara körbar.

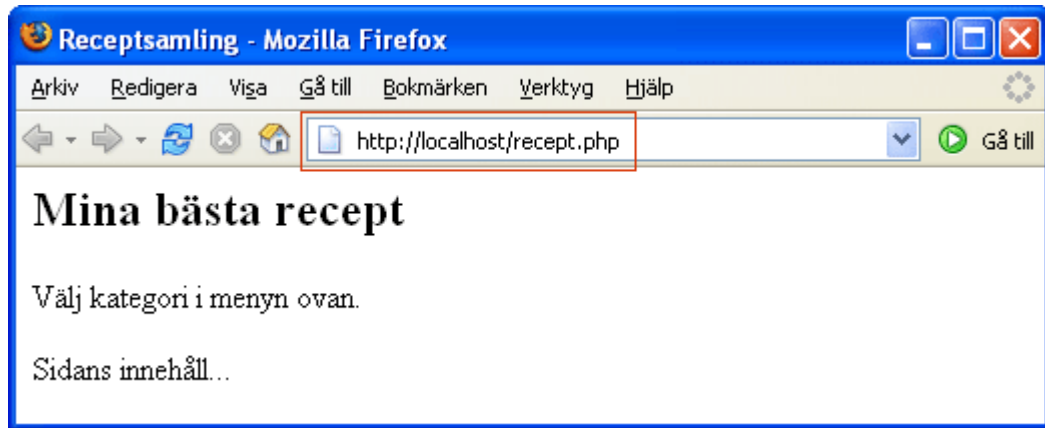
1. HTML-koden till sidan "recept.php" (som senare ska inkludera en meny):

```
<html>
<head>
<title>Receptsamling</title>
</head>
```

```
<body>

<h2>Mina bästa recept</h2>
<p>Välj kategori i menyn ovan.</p>
<p>Sidans innehåll... </p>
</body>
</html>
```

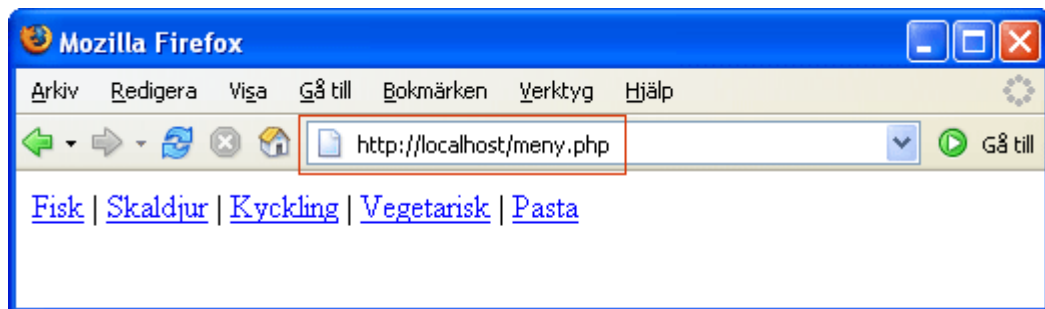
Filen "recept.php" visar sidinnehållet innan menyn infogats:



2. Filen "meny.php" innehåller **länkarna** men ingen övrig HTML-formatering:

```
<a href="fisk.htm">Fisk</a> | <a href="skaldjur.htm">Skaldjur</a> | <a
href="kyckling.htm">Kyckling</a> | <a href="vegetarisk.htm">Vegetarisk</a> | <a
href="pasta.htm">Pasta</a>
```

Menyn innan den infogats i de sidor den ska användas:



3. Infoga "meny.php" i sidan "recept.php" med funktionen **include**

```
<html>
<head>
<title>Receptsamling</title>
</head>
<body>

<?php include("meny.php"); ?>

<h2>Mina bästa recept</h2>
<p>Välj kategori i menyn ovan.</p>
<p>Sidans innehåll... </p>
</body>
</html>
```

Filen "recept.php" visar nu sidinnehållet och den infogade menyn:



Du kan även använda **require** som fungerar likadant som **include** men det som skiljer funktionerna åt är vad som sker när filen saknas eller annat fel finns i scriptet. Om du använder **include** så visas ett varningsmeddelande (warning) men scriptet fortsätter att köras. Om du använder **require** så stoppas körningen (fatal error) (gäller endast från PHP 4.3.5 och framåt). Vilken funktion du ska använda beror på vad scripten utför. Om du vill hämta en meny till sidan så kanske du ändå vill visa sidan även om menyn saknas men om du använder fler PHP-funktioner som är beroende av varandra är det kanske bättre att stoppa körningen helt tills felen är åtgärdade.

Sessioner

HTTP (HyperText Transfer Protocol) används för att skicka data på Internet och anses vara ett lägeslöst (stateless) protokoll som inte sparar information om dig som användare när du surfar vidare till andra websidor. Varje sidvisning är alltså oberoende av vilken sida som tidigare besökts. I vissa fall vill man ändå lagra information som kan flyttas mellan websidorna. Ett exempel är när du loggar in på en webbplats med användarID och lösenord som då gäller under hela ditt besök på alla sidor i den aktuella webbplatsen. Samma sak gäller för shoppingsystem där du lägger dina artiklar i en kundvagn och fortsätter att handla. Dina varor (och kunduppgifter) följer med dig mellan sidorna i butiken fram till kassan innan du lämnar butiken.

För att lagra data mellan websidor används cookies och sessioner. Cookies lagrar data en längre tid och är användbart för återkommande besök vid olika tidpunkter (läs mer om Cookies i nästa avsnitt).

Sessioner lagrar informationen under ett och samma besök när användaren förflyttar sig mellan de olika sidorna i webbplatsen. En session tilldelas ett unikt ID som lagras på webservern men även i användarens dator i form av en temporär cookie. Sessionen kan gälla från det att användaren loggar in på en webbplats och fram tills utloggning sker och sessionen avslutas. (Även formulär kan användas för att flytta information mellan sidor med "post" och "get" men då måste en submitknapp utföra åtgärden och du vill förmodligen bara ha den knappen när du använder ett formulär).

PHP har stöd för cookies från PHP 3.0 och för sessioner från PHP 4.0. OBS! Tidigare versioner använde `session_register()` för att lägga till variabler i en session men många webhotell tillåter inte längre `register_globals`. Från PHP 4.2.0 är `register_globals` dessutom avstängd (off) som standard och det rekommenderas istället att använda vektorn `$_SESSION` vilket visas i exemplen nedan.

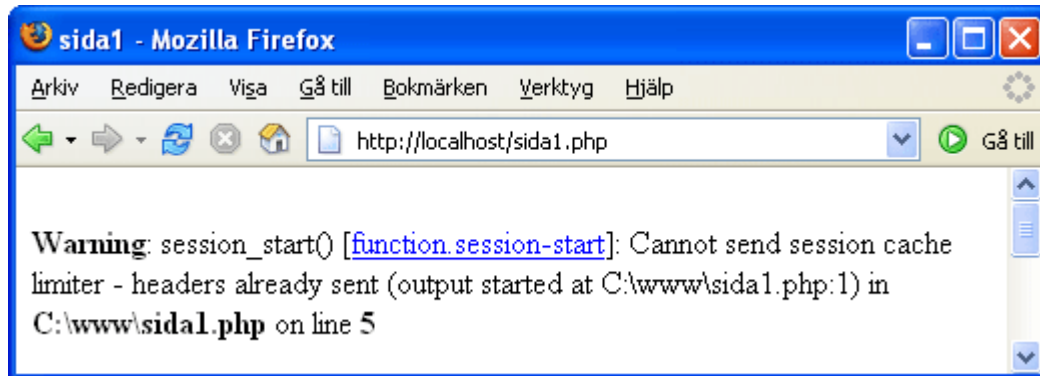
Innan en sida kan innehålla information i en session måste sessionshanteringen startas med funktionen `session_start()`. OBS! Koden måste vara **före** alla annan information som HTML-kod, text och övrig PHP-kod:

```
<?php
session_start();
?>
<html>
<body>
```

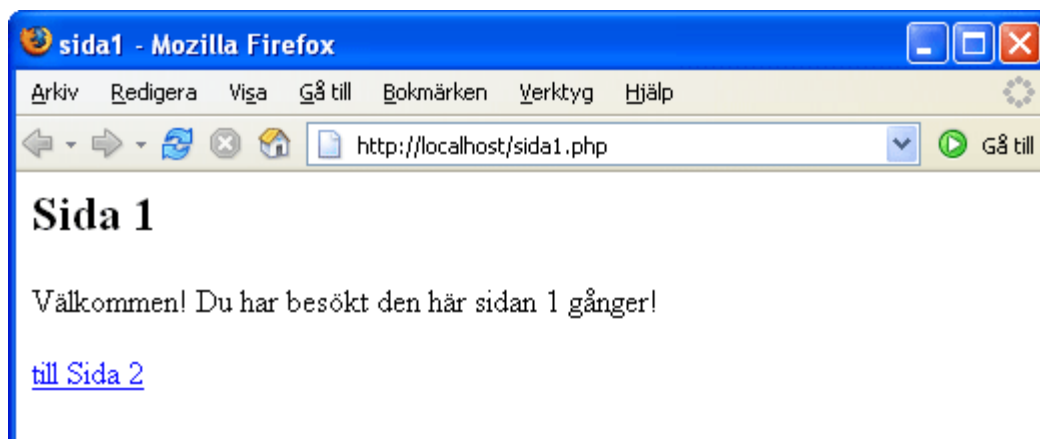
Sidans innehåll ...

```
</body>
</html>
```

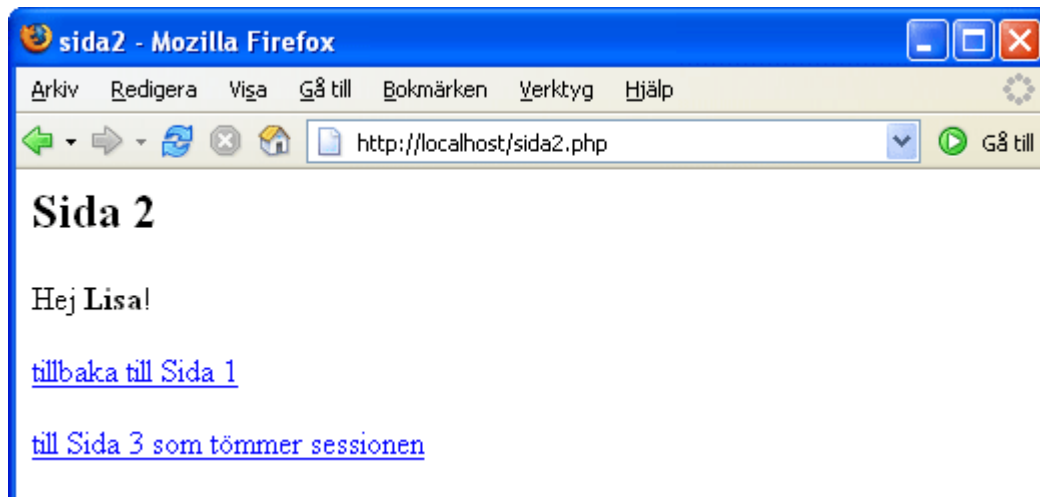
Inga tecken får finnas före kodavsnittet med sessionstarten och ett vanlig fel är att du av misstag gör ett **blanksteg** eller **radbytning** före inledningen av kodavsnittet vilket ger ett felmeddelande i webbläsaren:



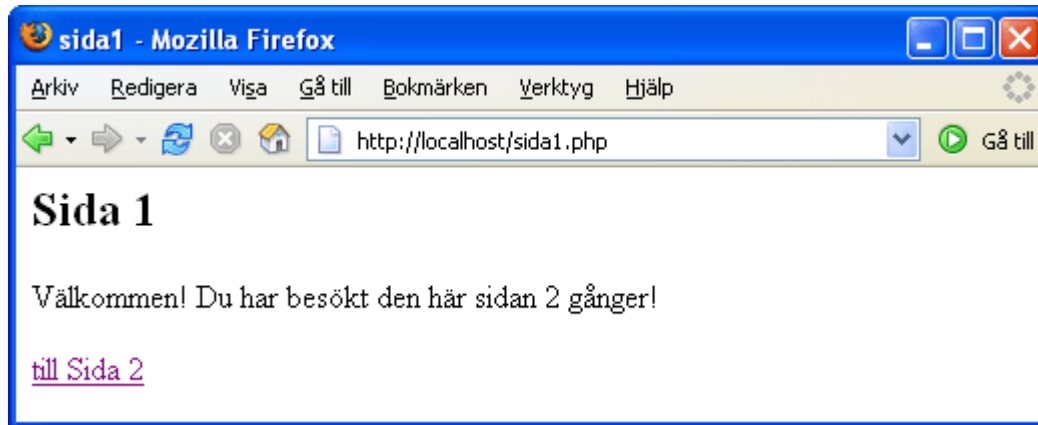
Som ett praktiskt exempel på session använder vi 3 sidor som länkar till varandra. När du kommer till Sida 1 så räknas **antal besök**:



... på Sida 2 visas ditt **medlemsnamn**:

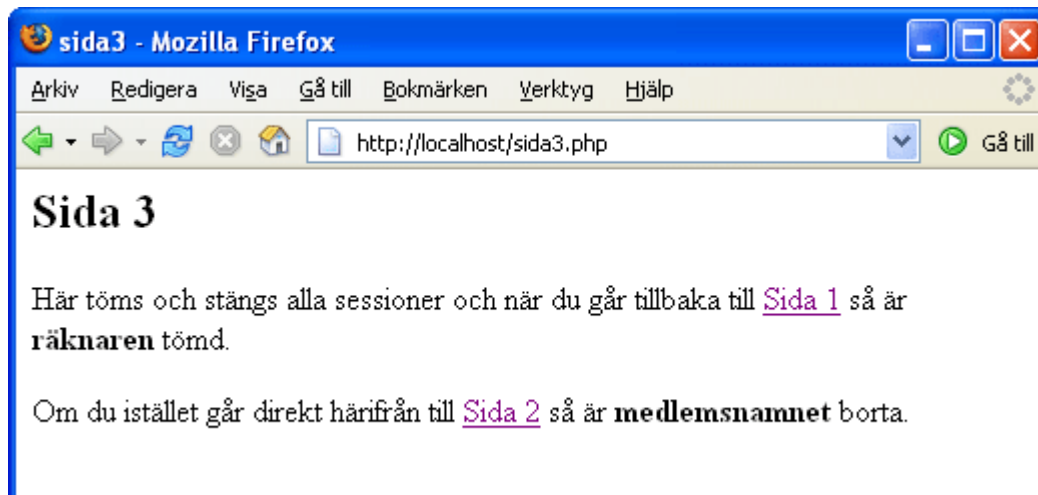


... om du använder länken (eller backar) tillbaka till Sida 1 så räknas ditt besök återigen:

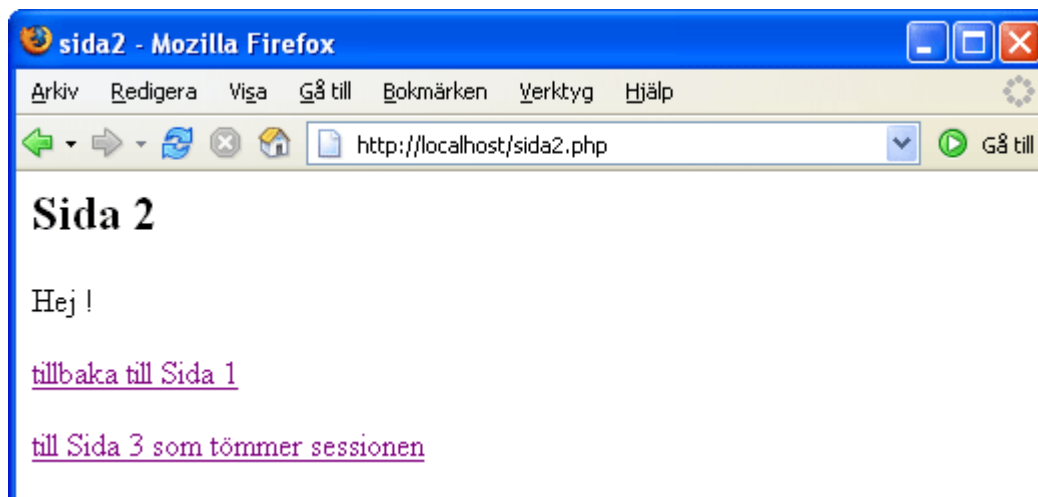


Här används alltså en **session** för att lagra dig som besökare när du förflyttar dig mellan sidorna i samma webbplats! Om du uppdaterar webbläsaren med **F5** eller går fram och tillbaka mellan sidorna ser du hur sessionen håller koll på antalet besök på Sida 1.

När du går till Sida 3 (via länken på sida 2) så **stängs sessionen** och all lagrad data raderas:



När sessionen är stängd och du återigen besöker Sida 1 börjar räknaren av dina dina besök från 1 igen och Sida 2 visar inte längre ditt medlemsnamn vid första besöket:



Observera att en ny session är ändå startad igen och pågår tills den avslutas igen dvs när du besöker

Sida 3 ännu en gång.

1. Här är kodexempel och beskrivning för de tre PHP-sidorna ovan:

sida1.php sida2.php sida3.php

Vårt exempel använder PHP-kod i HTML-sidor och viktigt är då att du anger PHP-koden som startar sessionen **före** all HTML-kod, text eller annan PHP-kod.

I filen "sida1.php" startar du sessionen med **session_start()** och en session gäller som standard i 180 minuter om inget annat anges.
Här vill vi att sessionen bara ska gälla i 30 minuter och anger det med **session_cache_expire(30)** som då anges **före session_start()**

Den superglobala variabeln **\$_SESSION** innehåller **nycklar** och **värden** i en array för att lagra antal sidvisningar och medlemsnamn. Varje session i PHP startas med funktionen **session_start()** som kontrollerar om en session redan existerar. Om sessionen finns återställs den och om den inte finns så skapas den. Den automatiska indexeringen i en array startar alltid med heltalet 0 om inget annat anges. I vårt exempel återställs sessionsvariabeln ["sidvisning"] som då ökas med värdet 1 varje gång sidan besöks:

\$_SESSION["sidvisning"]++

Här är hela koden för **sida1.php**:

```
<?php

// sessionen sparas i antal minuter
session_cache_expire(30);

// startar sessionen
session_start();

?>

<html>
<head><title>sida1</title></head>
<body>
<h2>Sida 1</h2>

<?php

// Ökning av angiven sessionsvariabel med 1
$_SESSION["sidvisning"]++;

// Namn på medlem sparas i sessionsvariabel
$_SESSION["medlem"] = "Lisa";

echo "Välkommen! Du har besökt den här sidan " . $_SESSION["sidvisning"] . " gånger!";

?>

<p><a href="sida2.php">till Sida 2</a></p>
</body>
</html>
```

2. Notera att sessionen måste startas igen på varje ny sida. Här gäller samma session för "sida2.php" om sessionen **startas upp** även här.

Arrayen **\$_SESSION["medlem"]** skrivs ut på sidan och innehållet är det medlemsnamn som angavs tidigare.

Här är hela koden för **sida2.php**:

```
<?php
session_start();

?>

<html>
<head><title>sida2</title></head>
<body>
<h2>Sida 2</h2>

<?php

// Hämtar sessionen med medlemsnamnet från Sida 1
echo "Hej <strong>" . $_SESSION["medlem"] . "</strong>!";

?>

<p><a href="sida1.php">tillbaka till Sida 1 </a></p>
<p><a href="sida3.php">till Sida 3 som tömmer sessionen </a></p>
</body>
</html>
```

3. Sessionen avslutas när webbläsaren stängs men kan stängas tidigare om tex en användare loggar ut. Innehållet i sessionsvariablerna **töms** med **session_unset()** och hela sessionen **stängs** med **session_destroy()**

Notera att sessionen först måste startas upp även här för att sedan kunna stängas.

Här är hela koden för **sida3.php**:

```
<?php
session_start();

?>

<html>
<head><title>sida3</title></head>
<body>
<h2>Sida 3</h2>

<?php

//Stänger alla öppna sessioner
session_unset();
session_destroy();

?>

<p>Här töms och stängs alla sessioner och när du går tillbaka till <a href="sida1.php">
Sida 1</a> så är <strong>räknaren</strong> tömd. </p>
<p>Om du istället går direkt härifrån till <a href="sida2.php">Sida 2</a> så är
<strong>medlemsnamnet</strong> borta.</p>
</body>
</html>
```

En session använder en temporär cookie som lagras i minnet och bara gäller så länge webbläsaren är aktiv. Sessionen avslutas och minnet töms när webbläsaren stängs. Om du vill ha en cookie som räcker längre måste den skrivas till användarens disk för att kunna hämtas vid nästa besök.

Cookies

En cookie används ofta till att identifiera en användare som återkommer till webbplatsen. Till skillnad från sessioner så lagras information mellan besöken och försvinner inte när webbläsaren stängs ned. Cookien innehåller variabelvärden som är länkade till en specifik domän och lagras i en textfil på användarens dator. Storleken på textfilen är högst 4 Kilobyte så det är alltså inga stora mängder information som kan sparas.

OBS! Enligt lagen om elektronisk kommunikation som trädde i kraft den 25 juli 2003, ska alla som besöker en webbplats med cookies få information om att webbplatsen innehåller cookies, vad dessa cookies används till och hur cookies kan undvikas. [Läs mer om den cookie-information som WDS använder här »](#)

Cookies sparas på användarens dator och du bör vara medveten om att vissa användare har stängt av cookie-funktionen i sin webbläsare. Anledningen är förmodligen att de är trötta på reklambanners och popupfönster som kan styras av användarinformationen som lagrats i en cookie-fil. Du bör alltså inte vara helt beroende av att cookies fungerar för att dina PHP-script ska fungera.

Funktionen **setcookie()** kan anges med flera argument som exemplet nedan visar:

setcookie(namn, värde, utgångsdatum/tid, sökväg, domän)

Namn och värde på cookien anges tillsammans med hur länge den ska finnas och existera. Om du inte anger någon utgångstid avslutas cookien när webbläsarens stängs (på samma sätt som en session). Sökvägen anger i vilken mapp (med undermappar) sidor som kan använda cookien får finnas. Om du bara vill att sidor i mappen "dindomän.se/shop" ska använda den angivna cookien anger du sökvägen "/shop/" och om du vill att alla filer i webbplatsen från rotmappen och nedåt ska använda den angivna cookien anger du "/" som sökväg. Anger du inget värde gäller samma mapp som dokumentet med den angivna cookien. Domännamnet som den angivna cookien ska gälla på anges med "dindomän.se" och gäller då även eventuella underdomäner som tex "shop/dindomän.se". Standardvärde om inget anges är samma värdnamn som den server som genererade cookien.

Här är ett exempel på hur funktionen med argumenten ovan kan användas:

```
<?php
setcookie("login", "username", time()+3600, "/", "webdesignskolan.se");
?>
```

Utgångstiden är given med funktionen **time()** och följs av tiden i **sekunder** som cookien ska existera. Ska cookien existera **en timme** kan du ange **time()+60*60** eller **time()+3600** och båda metoderna anger att cookien ska gälla en timme från nutid.

Ett dygn (24 timmar) kan anges **time()+86400** eller **time()+60*60*24** och **en månad** kan du ange som **time()+3600*24*30** eller **time()+60*60*24*30** eller **time()+86400*30** osv...

[Läs mer om tid- och datumfunktionen hos PHP.net »](#)

1. Det första exemplet använder en cookie för att räkna ut när det senaste besöket gjordes på sidan och använder tids/datum-funktionerna för att jämföra med nutid. Funktionen **isset()** används för att kontrollera om cookien är given och använder villkorssatserna **if** och **elseif** för att skriva meddelanden.

Här är koden till filen "cookies.php":

```
<?php
```

```
setcookie("last_visit", date("Y-m-d, G:i"), time()+3600*24*30);

?>

<html>
<head><title>Cookies</title></head>
<body>
<p>Finns det några kakor?</p>

<?php

if(isset($_COOKIE["last_visit"])) {

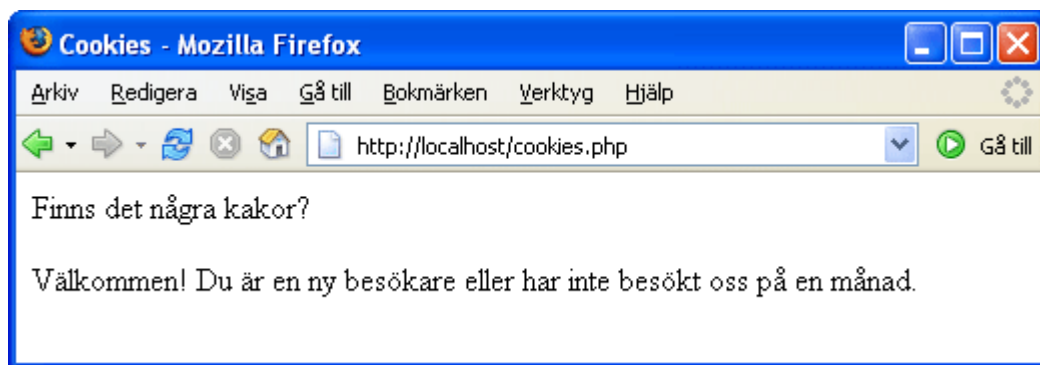
    echo "Ditt senaste besök här var: <br />" . $_COOKIE["last_visit"];
}

else {
    echo "Välkommen! Du är en ny besökare eller har inte besökt oss på en månad.";
}

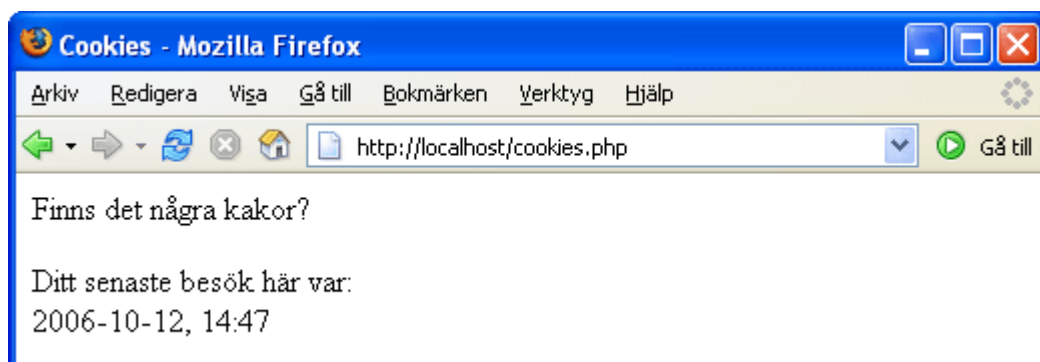
?>

</body>
</html>
```

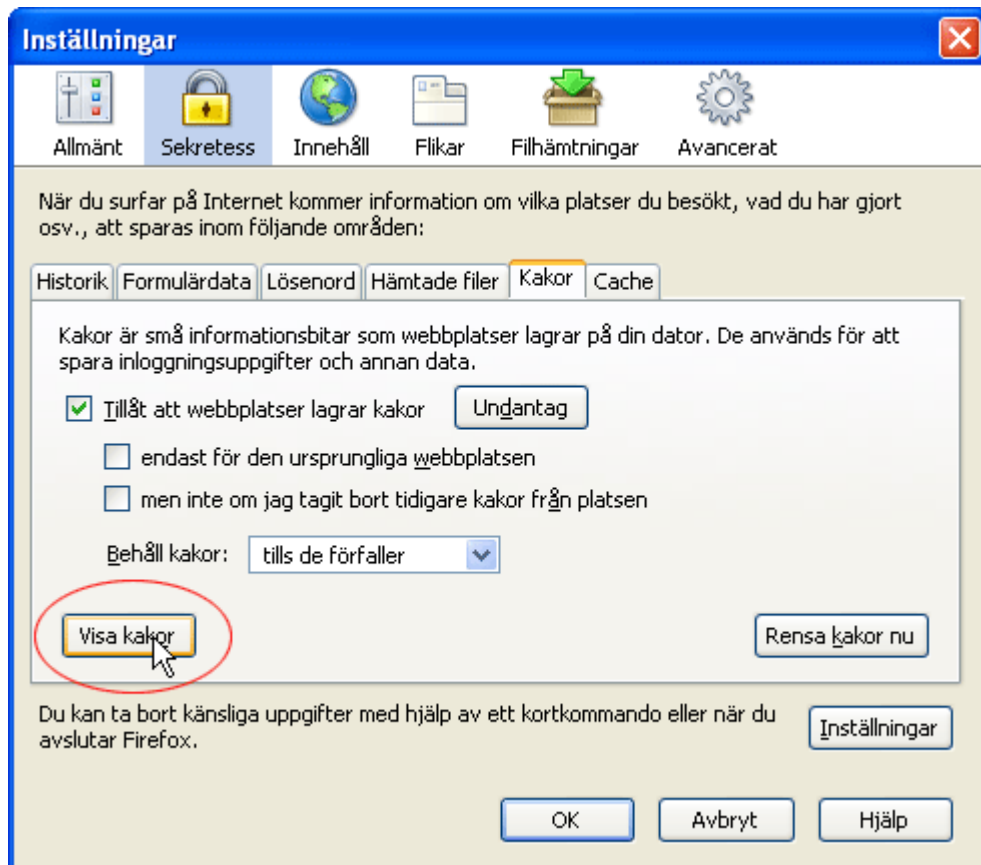
Filen "cookies.php" visas i webbläsaren första gången:



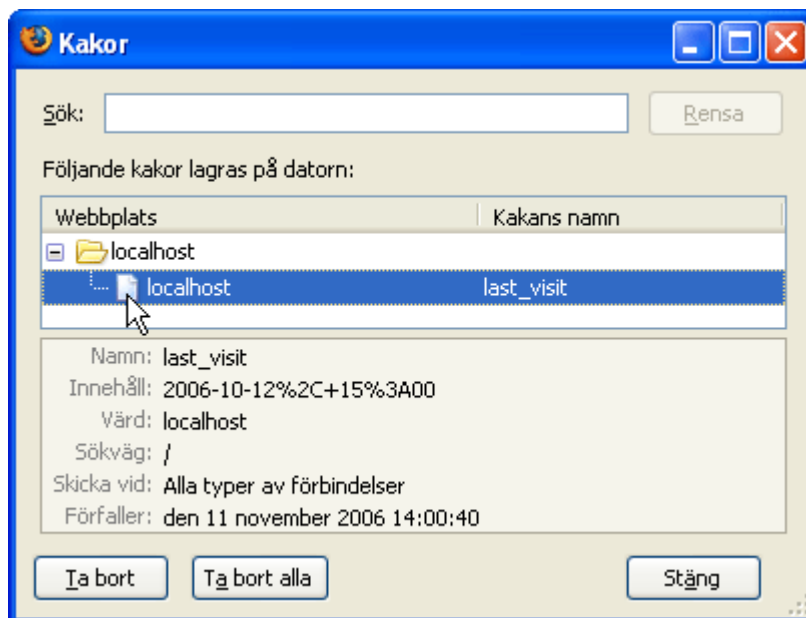
När du åter besöker sidan (eller uppdaterar med F5) visas tidpunkten för ditt senaste besök:



2. Kolla gärna om du kan se din cookie-fil! I **Firefox** använder du menyn "Verktyg/inställningar" och klickar på knappen "Visa kakor":



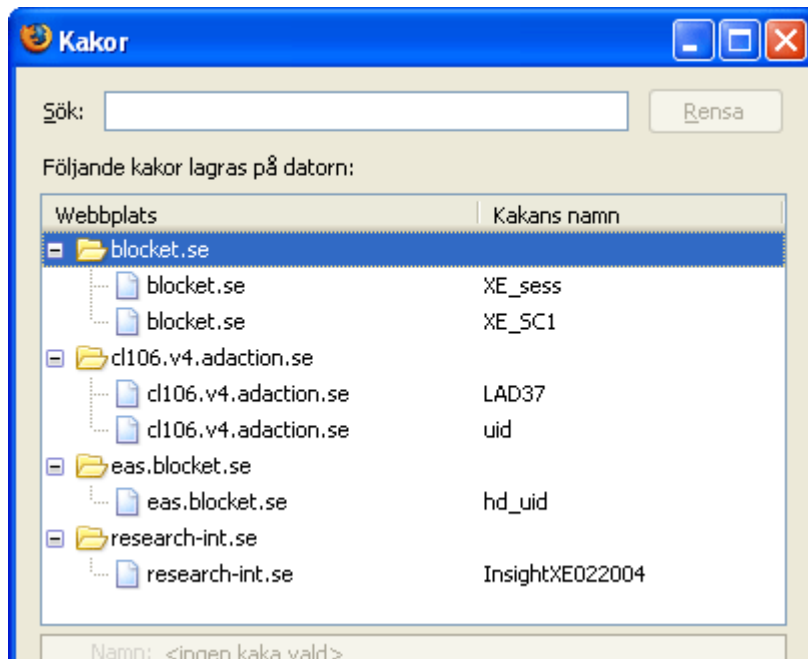
Här visas din cookie under domänen "localhost" när du testar dina PHP-filer på din lokala webserver:



Om du vill **ta bort din cookie** manuellt för att testa din PHP-fil igen så använder du knappen "Rensa kakor nu" eller "Ta bort alla" och uppdaterar sedan webbläsarenn på nytt med F5.

Om du använder Internet Explorer ser du hur du tar bort cookies i [WDS cookie-information »](#)

Cookies används frekvent av de flesta större webplatser och ett enda besök på www.blocket.se ger så här många cookies innan du ens hunnit välja ort...



Några cookies är temporära och försvinner när webbläsaren stängs men några finns kvar till den angivna utgångstiden.

3. Vårt andra exempel använder ett **formulär** för att flytta information mellan två sidor med "post" som action.

Här är koden till sidan med formuläret "medlemssidan.htm":

```
<html>
<head><title>Medlemssidan</title></head>
<body>
<h2>Medlemssidan</h2>
<p>Välkommen till medlemssidan! Här anger du ditt namn innan du går vidare till vår
mötesplats:</p>

<form action="http://localhost/set_cookie.php" method="post">
Namn:
<input name="namn" type="text">
<br>
<input name="skicka" type="submit" value="Gå vidare">
</form>

</body>
</html>
```

Formulärsidan "medlemssidan.htm" (som inte fungerar ännu):



4. Filen "set_cookie.php" tar emot informationen i formulärfältet "**namn**" och använder det som värde i cookie "medlem". Här används alltså inte funktionen `setcookie()` från början utan bara om ingen cookie med det angivna namnet finns (men koden måste ändå inleda dokumentet). Funktionen `isset()` (not isset) används för att kontrollera om cookien är angiven:

```
<?php

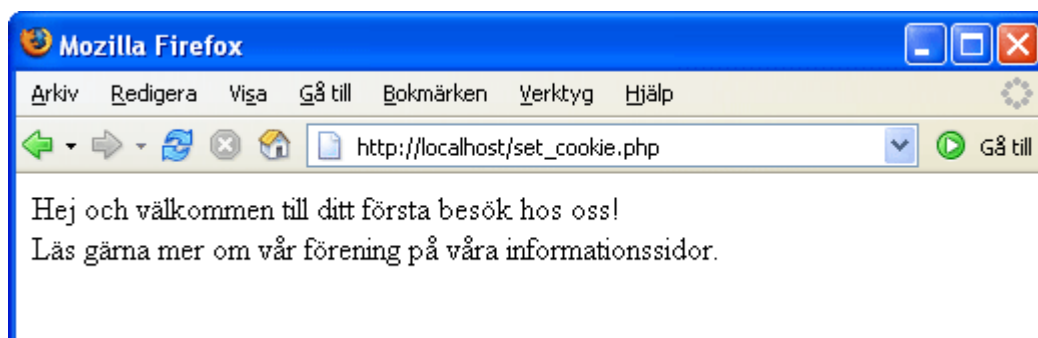
// Om en cookie INTE finns så skapas den här
if (!isset($_COOKIE["medlem"])) {

    setcookie("medlem", $_POST["namn"], time()+86400, "/");

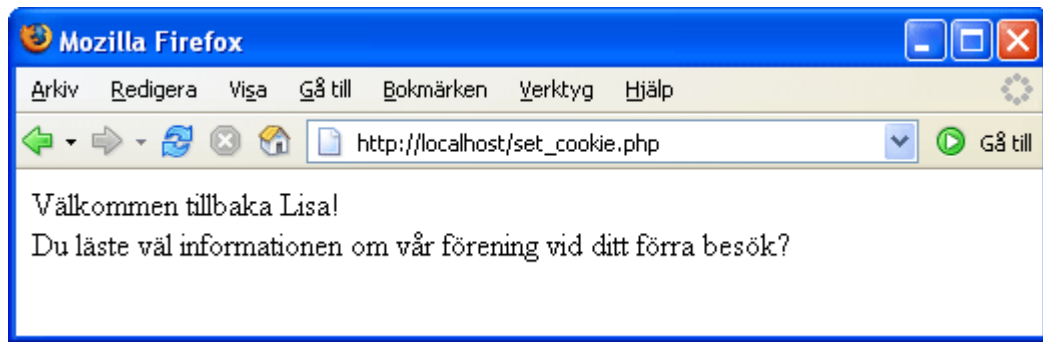
    echo "Hej och välkommen till ditt första besök hos oss! <br/> Läs gärna mer om vår
        förening på våra informationssidor.";
}
else {

    echo "Välkommen tillbaka " . $_POST["namn"] . "! <br /> Du läste väl
        informationen om vår förening vid ditt förra besök?";
}
?>
```

Första besöket på sidan "set_cookie.php" **skapar** cookie-filen och visar aktuellt meddelande:



När du åter besöker sidan (eller uppdaterar med F5) visas ditt namn och den andra meddelandetexten:



I vissa fall vill du **radera** en cookie-fil "manuellt" och det kan tex gälla ett inloggningssystem där cookien ska försvinna vid utloggningen. För att ta bort en cookie anger du samma funktion som startade den men med ett utgångsdatum som redan har passerat. Här anges att cookien ska starta och raderas omedelbart:

```
<?php  
  
setcookie("medlem", "", time()-3600);  
  
?>
```

Tiden anges till en timme i exemplet ovan men du kan ange vilken tid du vill med ett negativt tal.

Nästa steg för att använda PHP-effektivt är att använda databaser och du kan läsa mer om det i guiden [MySQL och databaser »](#)

[Copyright © **Webdesignskolan**
Materialet får skrivas ut och användas för personligt bruk.
Användning i undervisningssyfte är ej tillåten utan vårt tillstånd - [läs mer här »](#)]